

CV3: Registračný formulár v Jetpack Compose - Teória

1 Cieľ cvičenia

V tomto cvičení sa naučíme pracovať so vstupnými komponentmi v Jetpack Compose. Po cvičení by ste mali rozumieť tomu, ako sa vytvára formulár, ako sa spracúva vstup od používateľa, ako sa validujú údaje a ako sa používateľskému rozhraniu odovzdáva stav cez parametre.

Výsledkom bude jednoduchá registračná obrazovka s poľami pre meno, e-mail, heslo, potvrdenie hesla a súhlas s podmienkami. Aplikácia bude vedieť zobrazíť chyby pri nesprávnom vstupe, aktivovať tlačidlo až po splnení podmienok a po úspešnej registrácii zobrazí výslednú obrazovku.

2 State vo formulároch

Ako sme si už spomínali minulé cvičenie, každé vstupné pole musí mať svoju aktuálnu hodnotu uloženú v stave. Pre meno, e-mail, heslo alebo checkbox teda typicky používame samostatné premenné, prípadne jednu spoločnú dátovú štruktúru.

```
var fullName by remember { mutableStateOf("") }  
var email by remember { mutableStateOf("") }  
var acceptedTerms by remember { mutableStateOf(false) }
```

Keď sa zmení hodnota niektorého poľa, Compose spustí recomposition a automaticky prekreslí tú časť používateľského rozhrania, ktorá od tohto stavu závisí.

3 OutlinedTextField

Na vstup textu použijeme `OutlinedTextField`. Tento komponent patrí do Material 3 a je vhodný pre formuláre, pretože má jasný vizuálny rámček, podporuje label a vie zobrazovať chybový stav.

```
OutlinedTextField(  
    value = email,  
    onChange = { email = it },  
    label = { Text("Email") }  
)
```

Najdôležitejšie parametre `OutlinedTextField` sú:

- `value` - aktuálna hodnota poľa,
- `onValueChange` - funkcia, ktorá sa vykoná pri zmene textu,
- `label` - popis poľa,
- `isError` - prepínač chybového stavu,
- `keyboardOptions` - nastavenie typu klávesnice,
- `visualTransformation` - spôsob zobrazenia textu, napríklad skrytie hesla.

4 Validácia formulára

Validácia znamená kontrolu správnosti zadaných údajov. V tomto cvičení môžeme použiť jednoduché pravidlá. Meno nesmie byť prázdne, e-mail musí obsahovať znak @, heslo musí mať aspoň šesť znakov, potvrdenie hesla sa musí zhodovať s pôvodným heslom a používateľ musí potvrdiť súhlas s podmienkami.

```
val emailError = email.isBlank() || !email.contains("@")
val passwordError = password.length < 6
```

Takéto výrazy sa často označujú ako odvodený stav. Nie je potrebné ich držať v samostatnej `mutableStateOf` premennej, pretože ich vieme vždy vypočítať z aktuálnych hodnôt formulára.

5 Chybové hlášky a chybový stav

Je dôležité používateľa upozorniť na chybné údaje. Ak nastavíme `isError = true`, pole sa začne správať ako chybové. Zároveň môžeme pod pole zobraziť vysvetľujúci text. Takto sa používateľ okamžite dozvie, čo zadal nesprávne.

```
if (emailError) {
    Text(
        text = "Email must contain the @ symbol.",
        color = MaterialTheme.colorScheme.error
    )
}
```

6 Checkbox a boolean stav

Nie všetky vstupy vo formulári sú textové. Napríklad pri súhlase s podmienkami používame `Checkbox`, ktorý pracuje s logickou hodnotou `true/false`. Každý komponent číta stav a pri zmene ho posúva späť cez callback.

```
Checkbox(  
    checked = acceptedTerms,  
    onCheckedChange = { acceptedTerms = it }  
)
```

7 Zobrazenie a skrytie hesla

Pri heslách sa často používa `PasswordVisualTransformation()`, ktorá nahradí znaky bodkami alebo hviezdikami. Ak chceme heslo zobraziť, môžeme prepnúť transformáciu na `VisualTransformation.None`.

```
val passwordTransformation = if (showPasswords) {  
    VisualTransformation.None  
} else {  
    PasswordVisualTransformation()  
}
```

8 Enabled stav tlačidla

Tlačidlo registrácie má byť dostupné iba vtedy, keď je formulár správne vyplnený. To riešime pomocou parametra `enabled`.

```
Button(  
    onClick = onRegisterClick,  
    enabled = isValidForm  
) {  
    Text("Register")  
}
```

9 State hoisting

Pri menších ukázkach môžeme držať všetok stav priamo v jednej composable funkcii. Je však dôležité ukázať aj princíp *state hoisting*. To znamená, že stav držíme vo vyššej composable funkcii a nižšie komponenty dostanú iba hodnotu a callback na zmenu. Takéto riešenie je čitateľnejšie, lepšie sa testuje a neskôr sa veľmi prirodzene prepája s ViewModelom.

```
FormTextField(  
    value = uiState.email,  
    onChange = onEmailChange,  
    label = "Email",  
    isError = emailError,  
    errorMessage = "Email must contain the @ symbol."  
)
```

10 Rozdelenie UI na menšie composables

Aj keď je výsledná aplikácia stále pomerne malá, je vhodné rozdeliť ju na menšie časti. V tomto cvičení môžeme obrazovku rozdeliť napríklad na:

- RegistrationScreen,
- RegistrationFormContent,
- FormTextField,
- ShowPasswordRow,
- TermsRow,
- RegistrationSuccessScreen.

Takéto rozdelenie pripravuje študentov na ďalšie cvičenie, kde sa už budeme cielene venovať znovu použiteľnosti a rozkladu používateľského rozhrania na menšie komponenty.

11 Preview

Aj pri formulári je užitočné vytvoriť `@Preview`. Umožní nám rýchlo overiť vzhľad bez spustenia celej aplikácie.

```
@Preview(showBackground = true)  
@Composable  
fun RegistrationScreenPreview() {  
    MaterialTheme {  
        RegistrationScreen()  
    }  
}
```

Je vhodné vytvoriť aj preview pre výslednú obrazovku po registrácii, aby sme videli oba základné stavy aplikácie.